



5. Sincronização

5.1 Visão Geral da Sincronização

O sistema possui dois fluxos de sincronização distintos:

Fluxo	Direção	Descrição
Download (Pull)	Servidor → App	Busca dados atualizados da API e persiste no banco local
Upload (Push)	App → Servidor	Envia dados criados/editados offline para a API

5.2 SyncScheduler — Sincronização Automática (Pull)

O `SyncScheduler` (`lib/services/sync/sync_scheduler.dart`) é responsável pela sincronização periódica de download:

- **Frequência:** A cada 1 minuto (`Timer.periodic(Duration(minutes: 1))`)
- **Lifecycle-aware:** Implementa `WidgetsBindingObserver` — para quando o app vai para background, retoma quando volta
- **Condição:** Só executa se o usuário estiver logado (`authProvider.isLoggedIn`)

Ordem de sincronização (Pull):

```
1 1. CustomerGroups → GET /api/customer-groups
2 2. Customers     → GET /api/customers
3 3. TaskTypes     → GET /api/task-types
4 4. Tags          → GET /api/tags
5 5. Questionnaires → GET /api/questionnaires
6 6. Tasks         → GET /api/tasks (por último, pois depende dos
   anteriores)
```

Os itens 1-5 são executados em paralelo (`Future.wait`), e Tasks é executado sequencialmente após.

5.3 SyncResolverService — Sincronização de Upload (Push)

O `SyncResolverService` (`lib/services/sync/sync_resolver_service.dart`) processa a fila de sincronização:

Fluxo:

```

1 1. Verifica conectividade (connectivity_plus)
2 2. Se offline → retorna sem fazer nada
3 3. Busca todos os itens com status='pending' da SyncQueue
4 4. Para cada item, despacha para o service específico baseado no campo
   'model'
5 5. Em caso de sucesso → deleta o item da fila
6 6. Em caso de erro → atualiza status='error', incrementa attempts,
   registra ErrorResponse

```

Dispatch por modelo:

model	Service	Endpoint API
customer	SyncCustomerService	POST /api/customers
task	SyncTaskService	POST /api/tasks
task_report_signature	SyncTaskReportSignatureService	POST /api/task-report-signatures
task_attachment	SyncTaskAttachmentService	POST /api/task-attachments
task_questionnaire	SyncTaskQuestionnaireService	POST/PUT /api/task-questionnaires

5.4 O que é Enviado para o Servidor

Cada item da `SyncQueue` contém:

- `model` : tipo do recurso
- `operation` : operação (`create` ou `update`)
- `payload` : JSON completo com os dados a serem enviados
- `localId` : ID local do registro para atualização pós-sync

Após sincronização bem-sucedida:

1. O `remoteId` retornado pelo servidor é salvo no registro local
2. O item é removido da `SyncQueue`

Exemplo com `SyncCustomerService` :

```

1 // 1. Envia POST /api/customers com o payload
2 // 2. Recebe responseData com o ID remoto
3 // 3. Atualiza o customer local com o remoteId

```

```

4  await customerRepository.updateCustomerById(customerData,
   item.localId);
5  // 4. Remove da fila
6  await syncQueueRepository.deleteById(item.id);

```

5.5 O que é Recebido do Servidor

No fluxo de Pull, cada endpoint retorna a lista completa de registros. O app faz upsert (insert or update) baseado no `remoteId`:

- Se já existe um registro com aquele `remoteId` → atualiza
- Se não existe → insere

Isso é implementado via `onConflict: DoUpdate(...)` nos DAOs:

```

1  Future<int> saveTask(TasksCompanion task) async {
2    return await into(tasks).insert(
3      task,
4      onConflict: DoUpdate((_) => task, target: [tasks.remoteId]),
5    );
6  }

```

5.6 Estratégia de Conflito

- **Modelo atual:** Last-write-wins (servidor sempre ganha no Pull)
- **No Pull:** Os dados do servidor sobrescrevem os dados locais quando o `remoteId` coincide
- **No Push:** Dados locais são enviados ao servidor; se o servidor aceita, o `remoteId` é salvo localmente
- **Não há merge de conflitos:** Se um registro foi editado localmente e no servidor, a última sincronização (Pull) sobrescreve

5.7 Controle de Estado da Fila

Status	Descrição
<code>pending</code>	Aguardando sincronização (estado inicial)
<code>error</code>	Falha na última tentativa de sincronização

Quando um item falha:

```

1  await syncQueueRepository.updateItemByLocalId(
2    SyncQueueCompanion(
3      status: Value('error'),
4      errorResponse: Value(errorMessage),
5      attempts: Value(currentAttempts + 1),
6      lastSyncTry: Value(DateTime.now()),
7    ),

```

5.8 Sincronização Manual (Tela de Sync)

A `SyncView` (`lib/views/sync_view.dart`) permite ao usuário:

- Ver quantos itens estão pendentes na fila
- Sincronizar tudo de uma vez (botão "Sincronizar Tudo")
- Sincronizar individualmente cada categoria (clientes, tarefas, tags, etc.)
- Ver detalhes dos itens pendentes (modal com status, tentativas, erros)
- Acompanhar o status visual de cada sincronização (pendente, sincronizando, concluído, erro)

5.9 Upload de Arquivos em Questionários

O `SyncTaskQuestionnaireService` tem lógica especial para upload de arquivos:

1. Antes de enviar o questionário, verifica se há respostas do tipo arquivo com `pending_upload = true`
2. Para cada arquivo pendente, faz upload via `FileUploadService.performUpload()`
3. Substitui a referência local pela referência remota no payload
4. Só então envia o questionário completo para a API